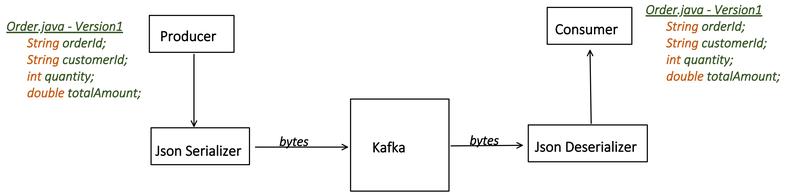
17. Schema Registry

Tuesday, 14 April 2026 11:14 AM

PROBLEM:

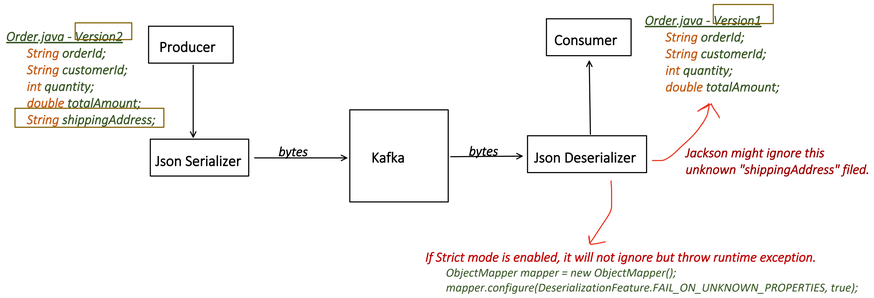


Till now, this is what we have done. And our producer and consumer is working fine.

But lets me show you how below 5 production use-cases can break this:

Usecase1: Addition of new field

- Producer upgraded to newer version of Order.java and added a new field *"shippingAddress"*
- Whereas Consumer is still on older version of Order.java, has no knowledge of *"shippingAddress"*



Usecase2: Removal of existing fields

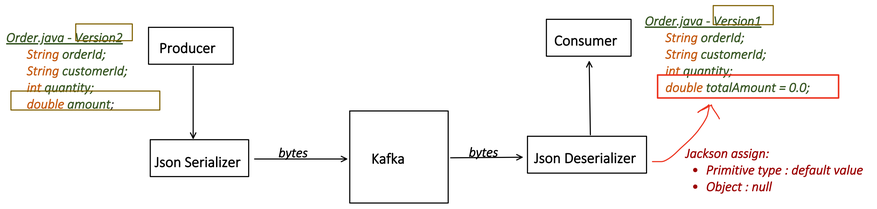
- Producer upgraded to newer version of Order.java and removed an old field *"totalAmount"*.
- Whereas Consumer is still on older version of Order.java and expect the field *"totalAmount"*.

real this field data is missing and should caught in validation.

This is too risky.

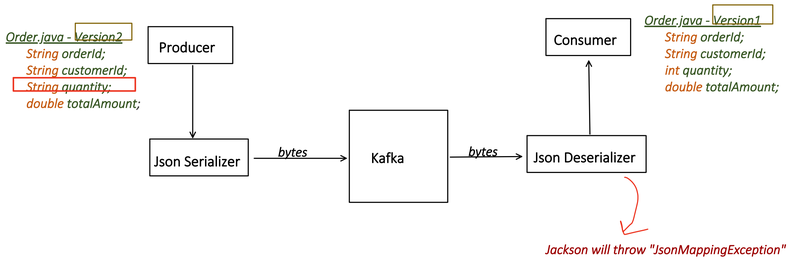
Usecase3: Rename a field

- Producer upgraded to newer version of Order.java and renames an old field from *"totalAmount"* to *"amount"*.
- Whereas Consumer is still on older version of Order.java and expect the field *"totalAmount"*.



Usecase4: Change of field type

- Producer upgraded to newer version of Order.java and changes *"quantity"* changes type from *"int"* to "String".
- Whereas Consumer is still on older version of Order.java and expect the field *"quantity"* type *"int"*



Usecase5: Multiple Consumers, different expectation

Consumer1 wants: all fields
Consumer2 wants: only orderId and totalAmount
Consumer3 wants: only orderId and ShippingAddress

In such scenario, this can create chaos when there is no Contract between producer and consumer on Event schema.

Because producer will keep changing or updating the event fields based on every consumer needs without knowing will this change breaks other consumer or not.

As JSON is schema-less and producer can send any JSON (without any version, validation or compatibility checks).

SOLUTION: Schema Registry

Schema : consider it as a formal definition of the structure of your data.

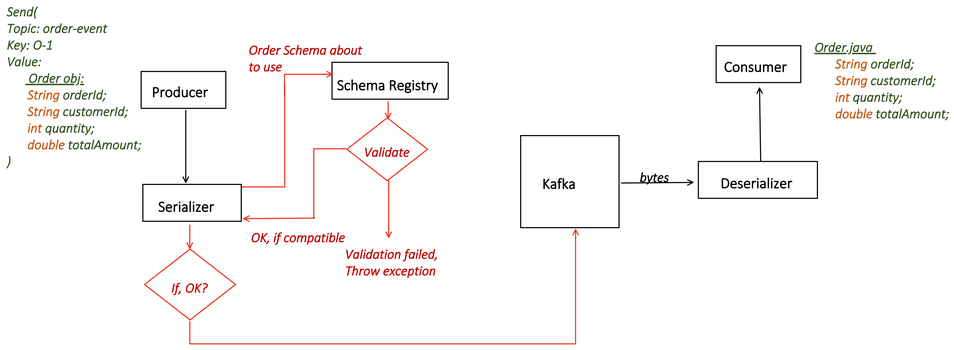
- Avro (most popular with Kafka)
- Protobuf

- JSON Schema

Schema Registry:

- Maintains Schema, its version.
- Any change in Schema performs compatibility check etc.

Very High level: Schema Registry brings validation before Producer send(or publish) an event.



BUT HOW? Lets go deeper:

Step1: Download Schema Registry and run

There are many providers like:

- Confluent Schema Registry (most popular)
- AWS Glue Schema Registry
- Azure Schema Registry

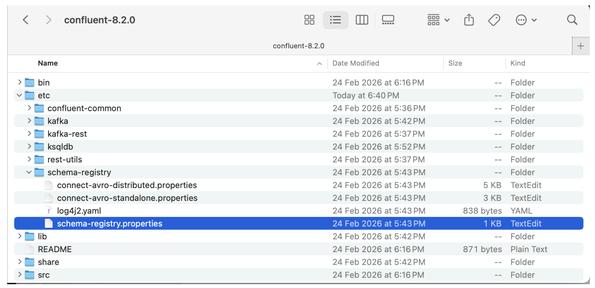
Etc..

I am proceeding with: Confluent Schema Registry (most popular)

download any latest version:

curl -O https://packages.confluent.io/archive/8.2/confluent-community-8.2.0.tar.gz

Open : confluent-8.2.0/etc/schema-registry/schema-registry.properties



Update schema-registry.properties:

```
#schema registry port number
```

```
listeners=http://0.0.0.0:8085

#bootstrap servers to connect with Kafka brokers
kafkastore.bootstrap.servers=PLAINTEXT://localhost:9092, PLAINTEXT://localhost:9192

kafkastore.topic=_schemas
```

That's after, starting the Controllers and Brokers, start the Schema Registry

```
/confluent-8.2.0/bin/schema-registry-start /confluent-8.2.0/etc/schema-registry/schema-registry.properties
```

Validate if Schema Registry is up and running by calling below, it should return []:

curl http://localhost:8085/subjects

Step2: Producer creates the Contract (Schema)

order.avsc

```
{
 "type": "record",
 "name": "Order",
 "namespace": "com.eda.schema",
 "fields": [
       { "name": "orderId", "type": "string" },
       { "name": "customerId", "type": "string" },
       { "name": "productId", "type": "string" },
       { "name": "quantity", "type": "int" },
       { "name": "totalAmount", "type": "double" }
     ]
}
```

In .avsc, we can define schema. It's a CONTRACT between producer and consumer

consider "record" as Object.
This .avsc will be used to create Java class with name Order

```
public class Order extends SpecificRecordBase {

    String orderId;
    String customerId;
    int quantity;
    double totalAmount;

    Schema getSchema() { . . . }

}
```

And its not a normal POJO,
It has additional functionalities like "getSchema", Which is used by Serializer.

Dependency we need to add for this is:

```
<dependency>
    <groupId>io.confluent</groupId>
    <artifactId>kafka-avro-serializer</artifactId>
    <version>${confluent.version}</version>
</dependency>

<dependency>
    <groupId>org.apache.avro</groupId>
    <artifactId>avro</artifactId>
    <version>${avro.version}</version>
</dependency>
```

- *Calls Schema Registry via Rest*
- *Serialization logic*

Parse Schema .avsc

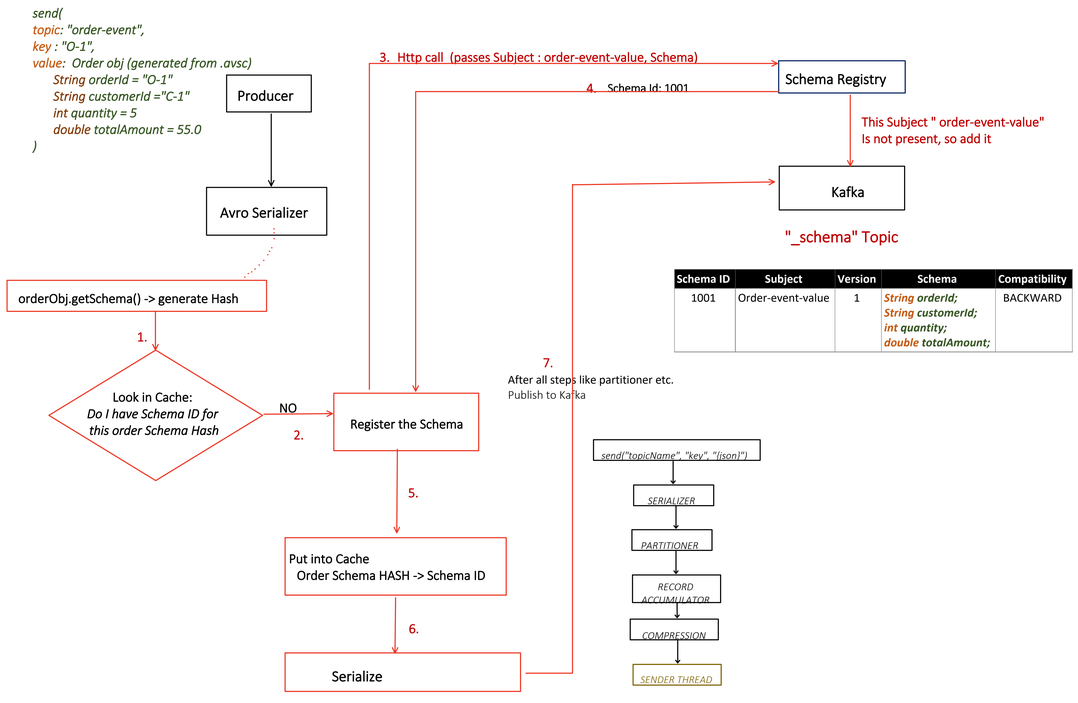
```
<plugin>
   <groupId>org.apache.avro</groupId>
   <artifactId>avro-maven-plugin</artifactId>
   <version>${avro.version}</version>
   <executions>
```

- *Generates Java code from .avsc*
- *Read from source Directory*
- *During compile, generates the Java code in output Directory*

```
      <execution>
        <phase>generate-sources</phase>
        <goals>
          <goal>schema</goal>
        </goals>
        <configuration>
          <sourceDirectory>${project.basedir}/src/main/avro</sourceDirectory>
          <outputDirectory>${project.build.directory}/generated-sources/avro</outputDirectory>
        </configuration>
      </execution>
    </executions>
  </plugin>
```

Step3: Producer makes first send() to Kafka

mvn clean compile : this will generate the Java Order class from .avsc



Schema ID	Subject	Version	Schema	Compatibility
1001	Order-event-value	1	*String orderId;* *String customerId;* *int quantity;* *double totalAmount;*	BACKWARD

[magic byte][schema ID][payload]

1 byte : Magic byte : always 0 (this tells the Deserializer whats the version of the wire protocol), with this deserializer know how to interpret rest of the bytes.

4 bytes : Schema ID
N bytes : Avro payload binary

Contains values in binary form (no fields)
Below is just for representation, internally it will be stored in binary form:
[0][1001][O-1 C-1 5 55.0]

Why this works?

Because with Schema ID, we can take out the schema, and we can get to know the fields and values (they are in order)

application.properties

```
server.port=8083
spring.application.name=schema-registry-producer


spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer

spring.kafka.producer.value-serializer=io.confluent.kafka.serializers.KafkaAvroSerializer

spring.kafka.producer.properties.schema.registry.url=http://localhost:8085
```

Step4: Producer makes second send() call but after changing the schema

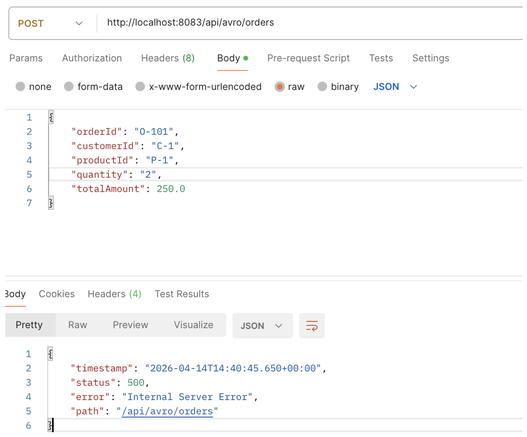
order.avsc

```
{
  "type": "record",
  "name": "Order",
  "fields": [
        { "name": "orderId", "type": "string" },
        { "name": "customerId", "type": "string" },
        { "name": "productId", "type": "string" },
        { "name": "quantity", "type": "string" },
        { "name": "totalAmount", "type": "double" }
      ]
}
```

Changed from int to string

mvn clean compile

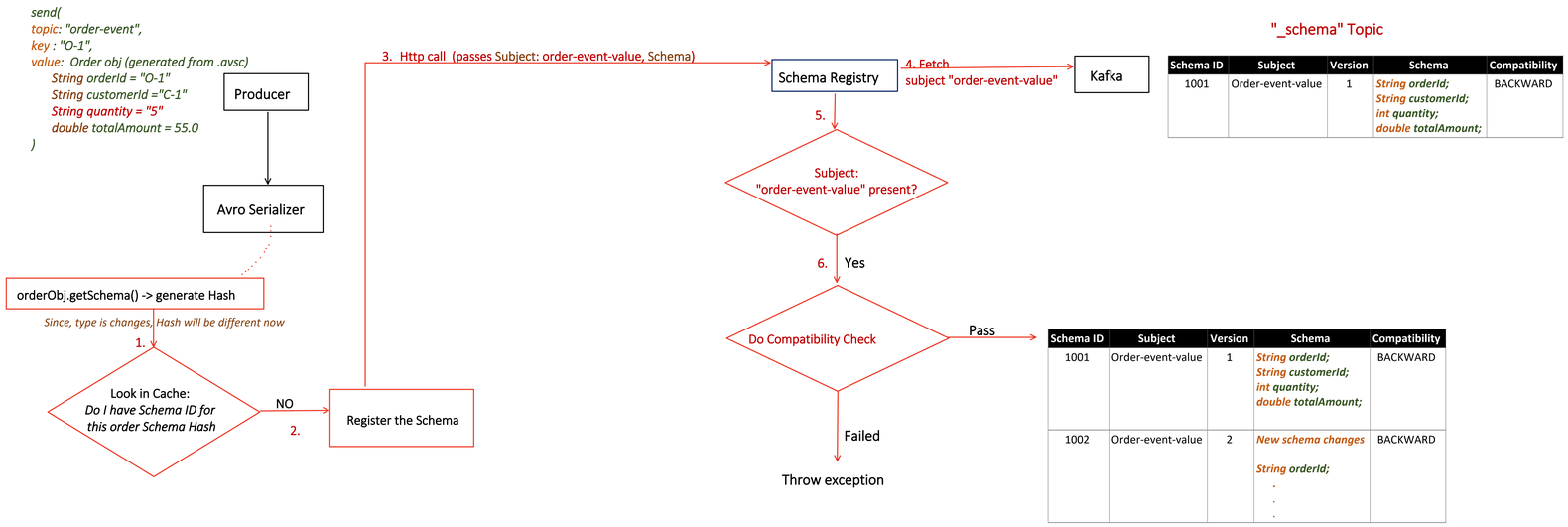
Lets try to publish the message again: now Producer will send the quantity in string



OUTPUT: Throws Exception

io.confluent.kafka.schemaregistry.client.rest.exceptions.RestClientException: Schema being registered is incompatible with an earlier schema for subject "order-events-value", details: [{errorType:'TYPE_MISMATCH', description:'The type (path '/fields/3/type') of a field in the new schema does not match with the old schema', additionalInfo:'reader type: STRING not compatible with writer type: INT'}, {oldSchemaVersion: 1}, {oldSchema: '{"type":"record","name":"Order","namespace":"com.eda.schema","fields":[{"name":"orderId","type":"string"},

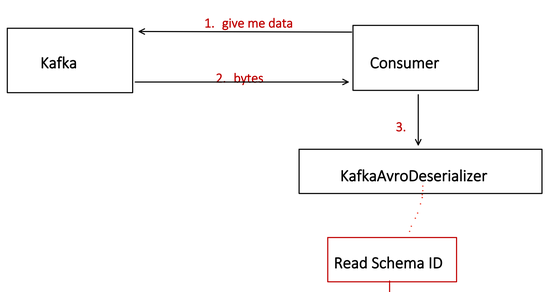
{"name":"customerId","type":"string"},{"name":"productId","type":"string"},{"name":"quantity","type":"int"},{"name":"totalAmount","type":"double"}]}'}, {validateFields: 'false',
compatibility: 'BACKWARD'}]; error code: 40901
    at io.confluent.kafka.schemaregistry.client.rest.RestService.sendHttpRequest(RestService.java:484) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.rest.RestService.lambda$httpRequest$0(RestService.java:656) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.rest.RetryExecutor.retry(RetryExecutor.java:53) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.rest.RestService.httpRequest(RestService.java:656) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.rest.RestService.registerSchema(RestService.java:889) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.rest.RestService.registerSchema(RestService.java:873) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.rest.RestService.registerSchema(RestService.java:865) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.CachedSchemaRegistryClient.registerAndGetId(CachedSchemaRegistryClient.java:403) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.CachedSchemaRegistryClient.registerWithResponse(CachedSchemaRegistryClient.java:547) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.schemaregistry.client.CachedSchemaRegistryClient.registerWithResponse(CachedSchemaRegistryClient.java:520) ~[kafka-schema-registry-client-8.2.0.jar:na]
    at io.confluent.kafka.serializers.AbstractKafkaSchemaSerDe.registerWithResponse(AbstractKafkaSchemaSerDe.java:555) ~[kafka-schema-serializer-8.2.0.jar:na]
    at io.confluent.kafka.serializers.AbstractKafkaAvroSerializer.serializeImpl(AbstractKafkaAvroSerializer.java:124) ~[kafka-avro-serializer-8.2.0.jar:na]
    at io.confluent.kafka.serializers.KafkaAvroSerializer.serialize(KafkaAvroSerializer.java:68) ~[kafka-avro-serializer-8.2.0.jar:na]
    at org.apache.kafka.clients.producer.KafkaProducer.doSend(KafkaProducer.java:1046) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.producer.KafkaProducer.send(KafkaProducer.java:993) ~[kafka-clients-3.9.1.jar:na]
    at org.springframework.kafka.core.DefaultKafkaProducerFactory$CloseSafeProducer.send(DefaultKafkaProducerFactory.java:1103) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.springframework.kafka.core.KafkaTemplate.doSend(KafkaTemplate.java:852) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.springframework.kafka.core.KafkaTemplate.observeSend(KafkaTemplate.java:820) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.sp

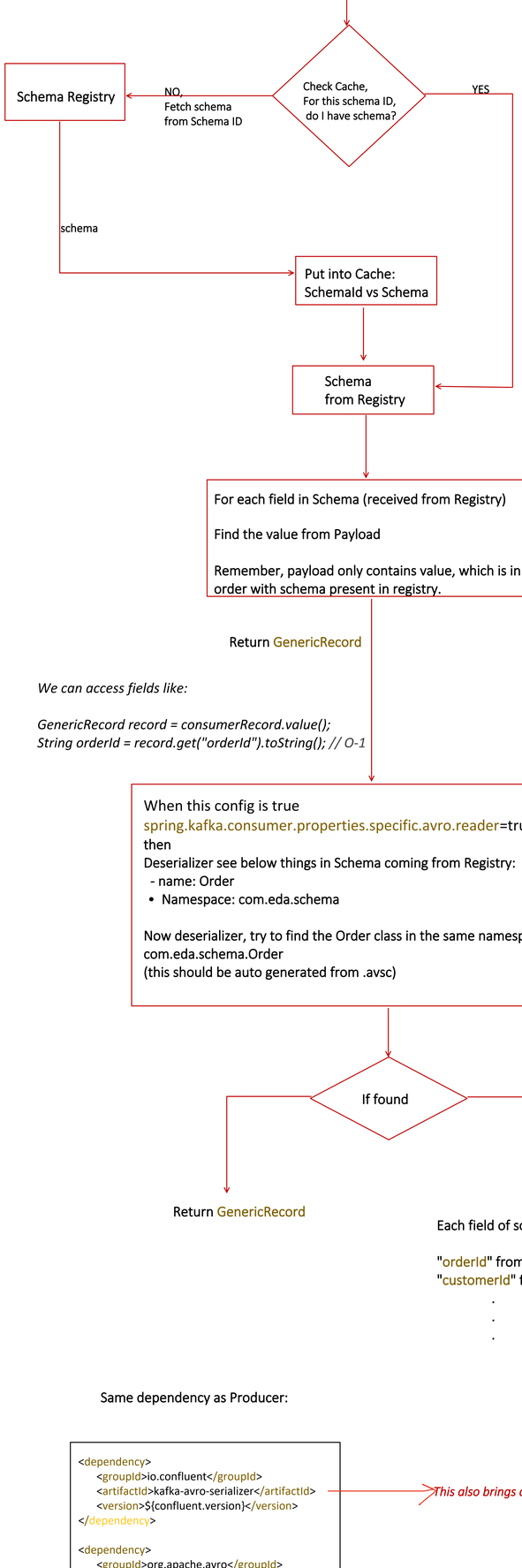


<u>One question might be coming, what is this Compatibility Check?</u>

Will cover it soon after complete flow (after consumer part too)

<u>Step5: Consumer read the event</u>





Order schema:
String orderId;
String customerId;
int quantity;
double totalAmount;

Payload in binary:
(this is conceptual, direct values I am writing)
[O-1 C-1 5 55.0]

Deserializer knows for string, how many bytes
I have to read, for int how many bytes likewise:

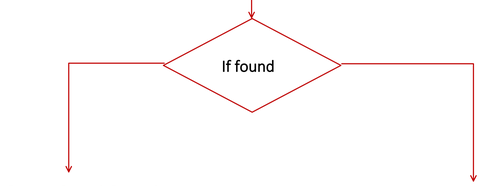
1st field : orderId : O-1
2nd field : customerId : C-1
3rd field : quantity : 5
4th field : totalAmount : 55.0

We can access fields like:

```
GenericRecord record = consumerRecord.value();
String orderId = record.get("orderId").toString(); // O-1
```

When this config is true
spring.kafka.consumer.properties.specific.avro.reader=true
then
Deserializer see below things in Schema coming from Registry:
- name: Order
- Namespace: com.eda.schema

Now deserializer, try to find the Order class in the same namespace.
com.eda.schema.Order
(this should be auto generated from .avsc)



Each field of schema coming from registry:

"orderId" from schema registry -> "orderId" Consumer Order schema
"customerId" from schema registry -> "customerId" Consumer Order schema
.
.
.

Same dependency as Producer:

```
<dependency>
    <groupId>io.confluent</groupId>
    <artifactId>kafka-avro-serializer</artifactId>
    <version>${confluent.version}</version>
</dependency>

<dependency>
    <groupId>org.apache.avro</groupId>
    <artifactId>avro</artifactId>
    <version>${avro.version}</version>
```

This also brings deserializer

```
</dependency>
```

```
<plugin>
  <groupId>org.apache.avro</groupId>
  <artifactId>avro-maven-plugin</artifactId>
  <version>${avro.version}</version>
  <executions>
    <execution>
      <phase>generate-sources</phase>
      <goals>
        <goal>schema</goal>
      </goals>
      <configuration>
        <sourceDirectory>${project.basedir}/src/main/avro</sourceDirectory>
        <outputDirectory>${project.build.directory}/generated-sources/avro</outputDirectory>
      </configuration>
    </execution>
  </executions>
</plugin>
```

application.properties

```
server.port=8084
spring.application.name=schema-registry-consumer

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-avro-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=io.confluent.kafka.serializers.KafkaAvroDeserializer


spring.kafka.consumer.properties.schema.registry.url=http://localhost:8085

#this is required, if we want GenericRecord to be converted to specific Java Object, else Avro Deserializer will return GenericRecord only
spring.kafka.consumer.properties.specific.avro.reader=true
```

order.avsc

```
{
 "type": "record",
 "name": "Order",
 "namespace": "com.eda.schema",
 "fields": [
       { "name": "orderId", "type": "string" },
       { "name": "customerId", "type": "string" },
       { "name": "productId", "type": "string" },
       { "name": "quantity", "type": "int" },
       { "name": "totalAmount", "type": "double" }
     ]
}
```

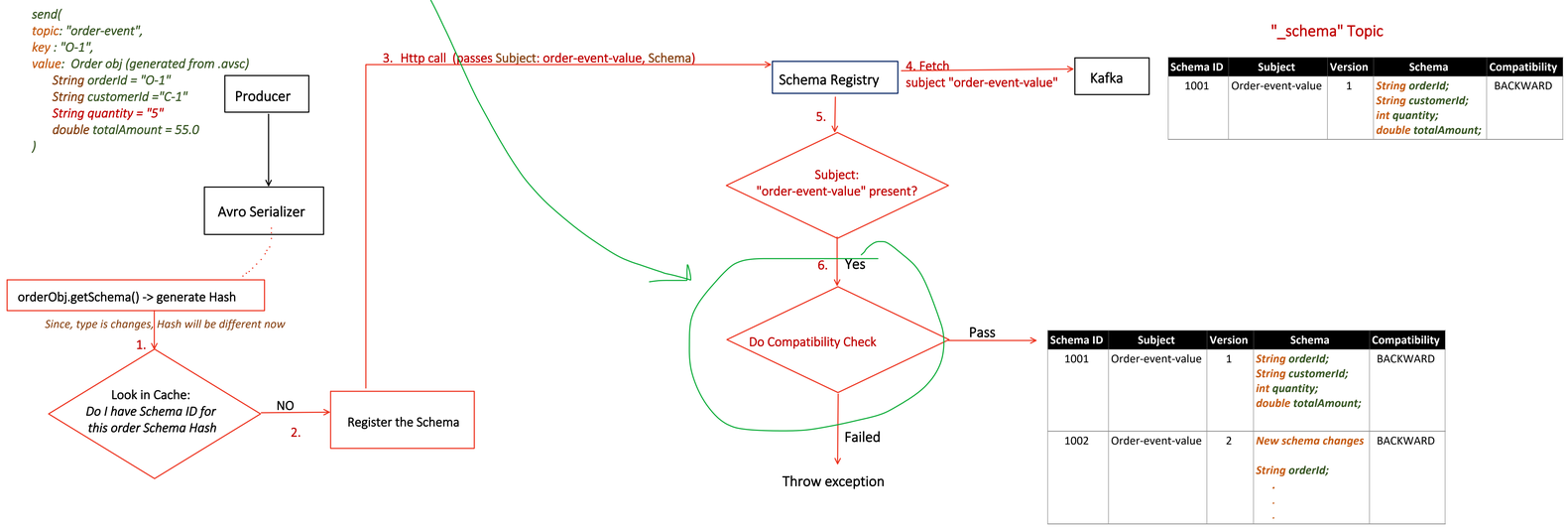
```
@Component
public class OrderAvroListener {

@KafkaListener(topics="order-events")
public void consumeOrder(ConsumerRecord<String, Order> record) {

        Order order = record.value();
        System.out.println("orderId:" + order.getOrderId());
    }
}
```

Now, we know the complete flow, let come back to the open question:

How Schema Registry do the validation, whether the event schema which producer is going to publish is compatible or not?



in "_schema" Topic, if you notice there is something called Compatibility.

Which can be:

- BACKWARD
- FORWARD
- FULL
- BACKWARD_TRASITIVE
- FORWARD_TRASITIVE
- FULL_TRASITIVE

Compatibility Level	Meaning	
BACKWARD (default)	Consumer with newer schema version -> can reads Old schema events Example, Let say: Schema Registry - V1 { "name": "orderId", "type": "int" } Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string", "default": "PENDING" } Now, Schema Registry will think from perceptive that: If Consumer on V2, pull the msg that was written with V1, there should not be any CRASH.	Usecase1: Added new fields with default Schema Registry - V1 { "name": "orderId", "type": "int" } Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string", "default": "PENDING" } Schema Registry think like: Old Kafka msgs: { order_id} Consumer with New Schema Version - V2: int orderId, String status This is SAFE, as in old schema msg status will be missing, so default value will be used. Usecase2: Add new fields without default Schema Registry - V1 { "name": "orderId", "type": "int" } Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string"} Not allowed, as in old schema "status" will be missing and default value is also missing.

		Usecase3: Remove field Schema Registry - V1 { "name": "orderId", "type": "int" } { "name": "status", "type": "string"} Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, This is SAFE, as old schema msg status will be extra, which will be ignored.
FORWARD	Consumer with Old schema -> can reads New schema events Example, Let say: Schema Registry - V1 { "name": "orderId", "type": "int" } Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string", "default": "PENDING" } Now, Schema Registry will think from perceptive that: If Consumer on V1, pull the msg that was written with V2, there should not be any CRASH.	Usecase1: Add a new fields Schema Registry - V1 { "name": "orderId", "type": "int" } Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string"} Schema Registry think like: New Kafka msg (V2): { order_id, status} Consumer with old Schema Version - V1: int orderId This is SAFE, as in new schema msg status will be extra, which consumer with old schema can ignore. Usecase2: Remove field with Default Schema Registry - V1 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string", default ="PENDING"} Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" }, This is SAFE, as in new schema msg status will be missing, which consumer with old schema expect but since its not present, it will use default. Usecase3: Remove field without Default Schema Registry - V1 { "name": "orderId", "type": "int" }, { "name": "status", "type": "string"} Producer is sending event with new Schema Schema Registry - V2 { "name": "orderId", "type": "int" } This is NOT ALLOWED, as in new schema msg status will be missing, which consumer with old schema expect but default value is also missing.
FULL (Backward + Forward)	Run the rules from both BACKWARD and FORWARD perceptive	
BACKWARD_TRASITIVE FORWARD_TRASITIVE FULL_TRASITIVE	DO NOT just compare with Last version, but all previous versions	

```
#schema registry port number
listeners=http://0.0.0.0:8085

#bootstrap servers to connect with Kafka brokers
kafkastore.bootstrap.servers=PLAINTEXT://localhost:9092, PLAINTEXT://localhost:9192

kafkastore.topic=_schemas

compatibility.level=FORWARD
```

```
}
```